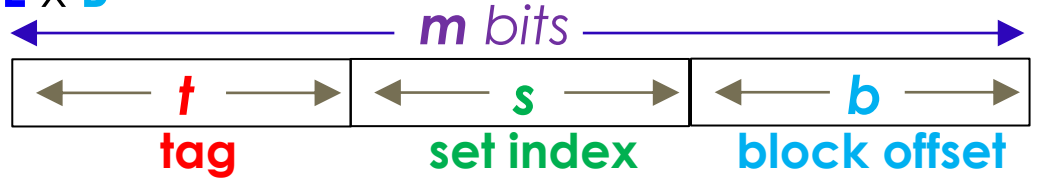
CMPT 295

Unit – Memory Hierarchy
Lecture 39 – Cache Memories – cont'd

Summary

- A **cache C** defined as (**S**, **E**, **B**, **m**) with memory of size of 2^m has:
  - **S** cache **sets**
  - Each **set** contains **E** cache lines
  - Each line contains:
    - 1 data block **B** of 2^b bytes, 1 valid bit **v**, **t** tag bits, where **t** = **m** – (**b** + **s**)
  - Capacity (size) of **C** = **S** x **E** x **B**
- Memory address:



- To find data in **L1 cache**:
  0. **Memory address expansion** (into bit pattern)
  1. **Set Selection:** Select cache **set** using **set index**
  2. **Line Matching:** If **v** valid, search **set** for matching **tag**

     -> if found: cache hit, -> if not found: cache miss
     - If cache miss occurs -> must search memory hierarchy at next lower level (expensive)
  3. **Word extraction:** When cache hit, **L1 cache** sends content of **block** at **block offset** to microprocessor

Today's Menu

- Cache-friendly code optimization:
  - Locality
  - Memory hierarchy
  - Caching
- Cache memories
  - How does a cache work?
  - How is cache structured?
  - Examples
  - Cache performance analysis
  - Another example of cache -> conflict miss
- Memory mountain

How to represent 3D arrays in memory (A9 Q. 3)

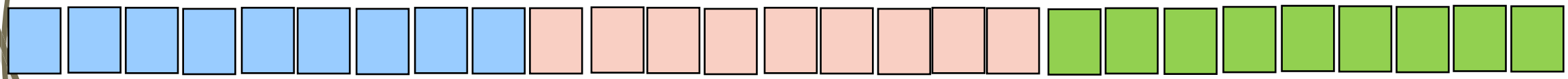
```
int sumArray(int A[N][N][N])
{
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += A[k][i][j];
    return sum;
}
```

- Let say N = 3 -> 3D array `A` in memory:
  - Start with first (outer) dimension -> row: **`A[k]`**

  - Then attend to the next dimension -> **`A[k][i]`**

  - Then attend to the innermost dimension -> **`A[k][i][j]`**

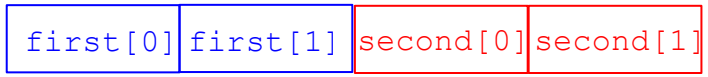


How **struct**'s are stored in memory

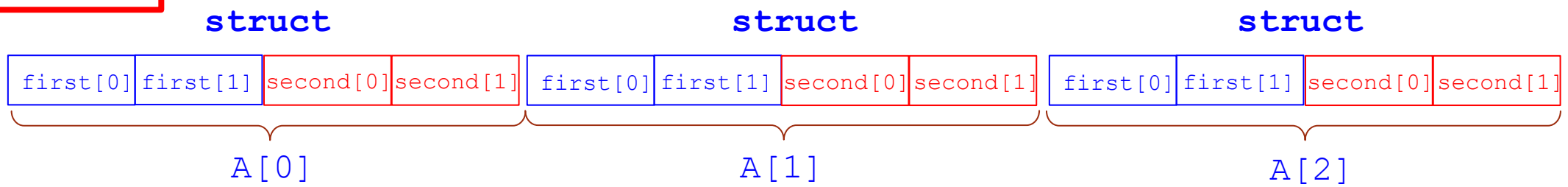
```
#define N 3
#define M 2
typedef struct {
    int first[M];
    int second[M];
} some_struct;

some_struct A[N];
```

1. **Start with the `struct` :**



2. **Then draw the actual array `A`:**



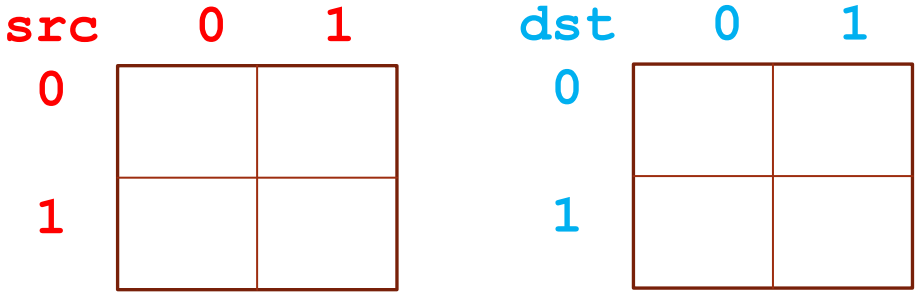
Let's not concern ourselves with alignment!

Let's do Practice Problem 6.17

- Memory model:
  - `sizeof(int)` = 4 bytes
  - **`src`** array starts at address 0 and **`dst`** starts at address 16 (decimal)
  - There is a single L1 that is direct-mapped with block size of 8 bytes
  - The cache has a total size of 16 bytes and the cache is initially empty
  - Accesses to **`src`** and **`dst`** arrays are the only sources of cache misses
- Question:
  - Indicate whether the access to **`src[row][col]`** and **`dst[row][col]`** is a cache hit (h) or a cache miss (m)?

```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10          }
11      }
12  }
```

The idea is:



`src`

	Col. 0	Col. 1
Row 0	m	_____
Row 1	_____	_____

`dst`

	Col. 0	Col. 1
Row 0	m	_____
Row 1	_____	_____

Let's start!

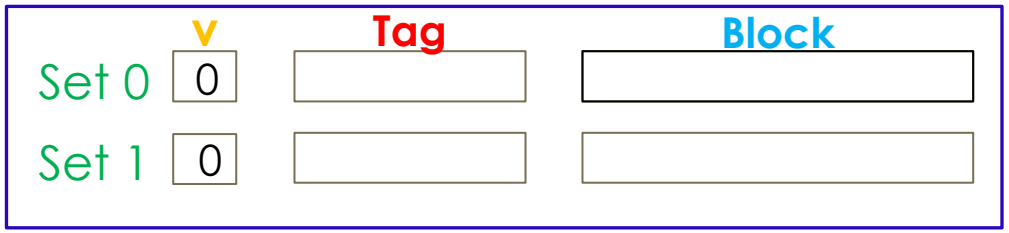
- Memory model:
  - `sizeof(int)` = 4 bytes
  - **`src`** array starts at address 0 and **`dst`** starts at address 16 (decimal)
  - There is a single L1 that is direct-mapped with block size of 8 bytes
  - The cache has a total size of 16 bytes and the cache is initially empty
  - Accesses to **`src`** and **`dst`** arrays are the only sources of cache misses
- Question:
  - Indicate whether the access to **`src[row][col]`** and **`dst[row][col]`** is a cache hit (h) or a cache miss (m)?

1. What do we know so far:
   - Memory hierarchy:

     CPU Regs -> L1 cache -> Main Memory
   - direct-mapped cache -> **E** = 1
   - Block size **B** -> 8 bytes => **b** = 3 bits
   - Cache size **C** = **S** * **E** * **B** => 16 bytes

     => **S** * 1line/set * 8 bytes/line = 16 bytes

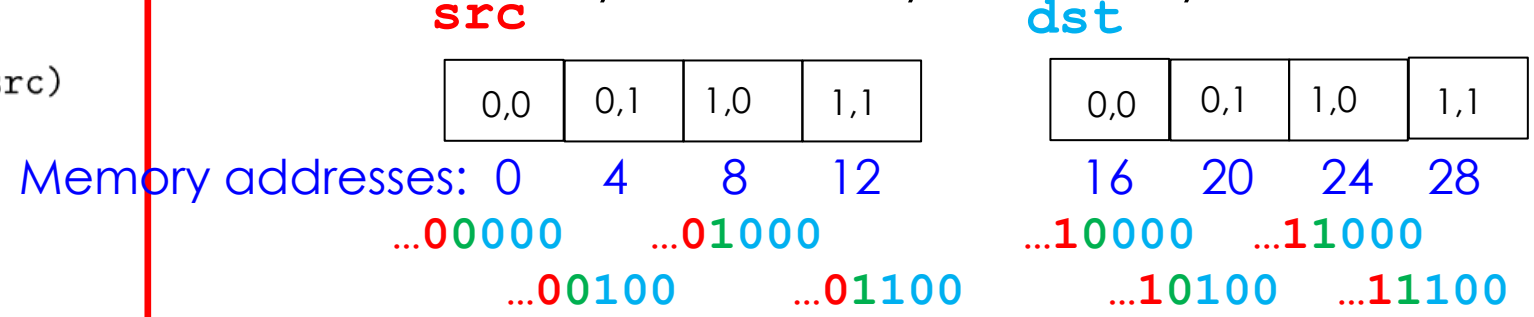
     => **S** = 2 sets => **s** = 1 bit **m** = ? bits
   - Initially -> cold cache (**v** bits -> 0)
   - Aside from **`src`** elements and **`dst`** elements, we are not accessing anything else from the "memory hierarchy"



```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10
11
12  }
```

2nd access: 0 0	1st: 0 0
4th access: 1 0	3rd: 0 1
6th access: 0 1	5th: 1 0
8th access: 1 1	7th: 1 1
column wise	row wise

2. Draw arrays in memory + memory addresses:



3. Partition the memory address into sections:
   tag (? bits) set index (1 bit) block offset (3 bits)
4. List the pattern of indices as elements are accessed by microprocessor => identify the **stride**!
5. Draw the cache …
6. Execute the code and update the cache:

	v	Tag	Block		v	Tag	Block
Set 0	0			Set 1	0		

The answer is:

src

	Col. 0	Col. 1
Row 0	m	
Row 1		

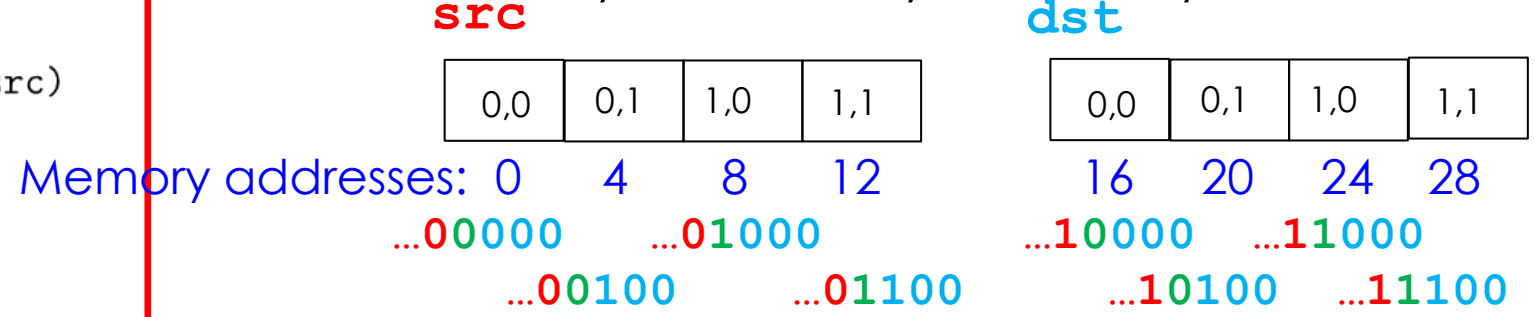
dst

	Col. 0	Col. 1
Row 0	m	
Row 1		

```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10
11
12  }
```

2nd access: 0 0	1st: 0 0
4th access: 1 0	3rd: 0 1
6th access: 0 1	5th: 1 0
8th access: 1 1	7th: 1 1
column wise	row wise

2. Draw arrays in memory + memory addresses:



3. Partition the memory address into sections:
   tag (? bits) set index (1 bit) block offset (3 bits)
4. List the pattern of indices as elements are accessed by microprocessor => identify the **stride**!
5. Draw the cache …
6. Execute the code and update the cache:

	v	Tag	Block		v	Tag	Block
Set 0	1	...0	src 0,0 src 0,1	Set 1	0		

The answer is:

src

	Col. 0	Col. 1
Row 0	m	
Row 1		

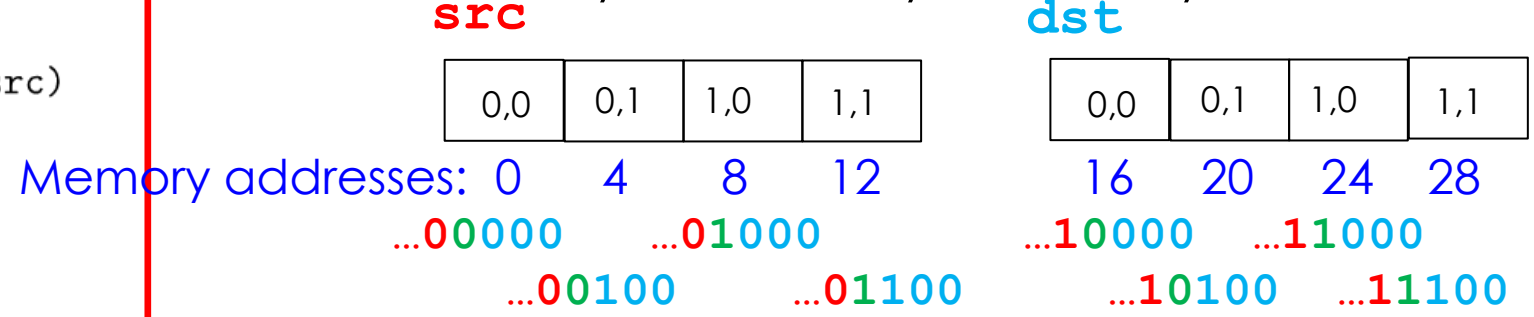
dst

	Col. 0	Col. 1
Row 0	m	
Row 1		

```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10
11
12  }
```

2nd access: 0 0	1st: 0 0
4th access: 1 0	3rd: 0 1
6th access: 0 1	5th: 1 0
8th access: 1 1	7th: 1 1
column wise	row wise

2. Draw arrays in memory + memory addresses:



3. Partition the memory address into sections:
   tag (? bits) set index (1 bit) block offset (3 bits)
4. List the pattern of indices as elements are accessed by microprocessor => identify the **stride**!
5. Draw the cache …
6. Execute the code and update the cache:

	v	Tag	Block		v	Tag	Block
Set 0	1	...1	dst 0,0 dst 0,1	Set 1	0		

The answer is:

src

	Col. 0	Col. 1
Row 0	m	
Row 1		

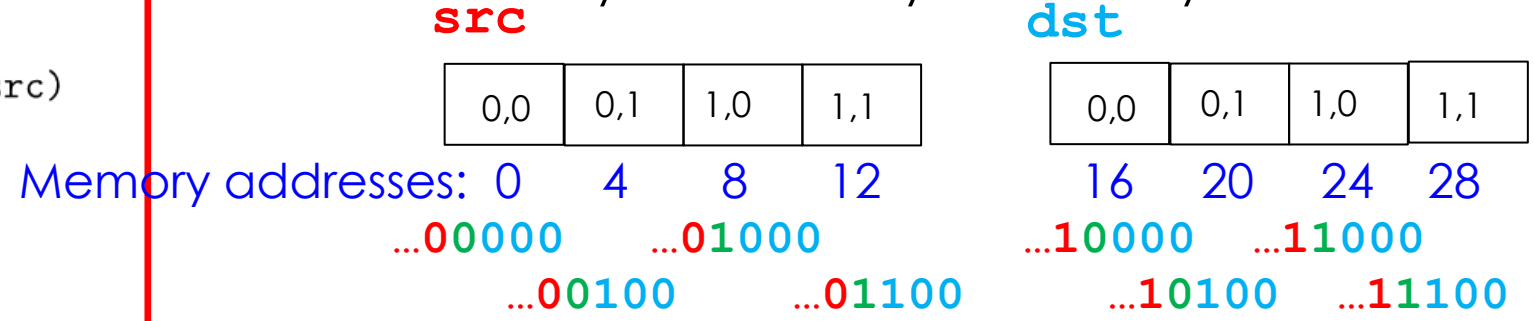
dst

	Col. 0	Col. 1
Row 0	m	
Row 1		

```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10
11
12  }
```

2nd access: 0 0	1st: 0 0
4th access: 1 0	3rd: 0 1
6th access: 0 1	5th: 1 0
8th access: 1 1	7th: 1 1
column wise	row wise

2. Draw arrays in memory + memory addresses:



3. Partition the memory address into sections:
   tag (? bits) set index (1 bit) block offset (3 bits)
4. List the pattern of indices as elements are accessed by microprocessor => identify the **stride**!
5. Draw the cache …
6. Execute the code and update the cache:

	v	Tag	Block		v	Tag	Block
Set 0	1	...0	src 0,0 src 0,1	Set 1	0		

The answer is:

src

	Col. 0	Col. 1
Row 0	m	
Row 1		

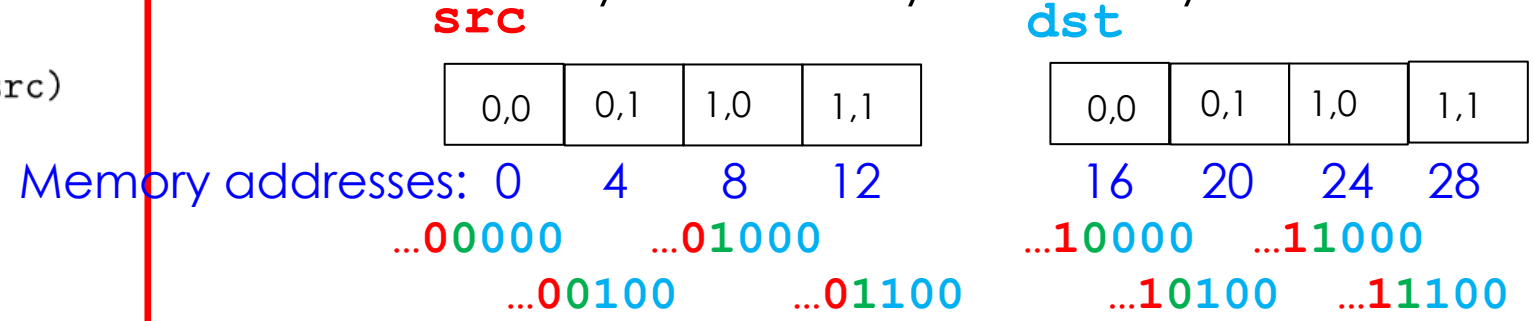
dst

	Col. 0	Col. 1
Row 0	m	
Row 1		

```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10
11
12  }
```

2nd access: 0 0	1st: 0 0
4th access: 1 0	3rd: 0 1
6th access: 0 1	5th: 1 0
8th access: 1 1	7th: 1 1
column wise	row wise

2. Draw arrays in memory + memory addresses:



3. Partition the memory address into sections:
   tag (? bits) set index (1 bit) block offset (3 bits)
4. List the pattern of indices as elements are accessed by microprocessor => identify the **stride**!
5. Draw the cache …
6. Execute the code and update the cache:

	v	Tag	Block		v	Tag	Block
Set 0	1	...0	src 0,0 src 0,1	Set 1	1	...1	dst 1,0 dst 1,1

The answer is:

src

	Col. 0	Col. 1
Row 0	m	
Row 1		

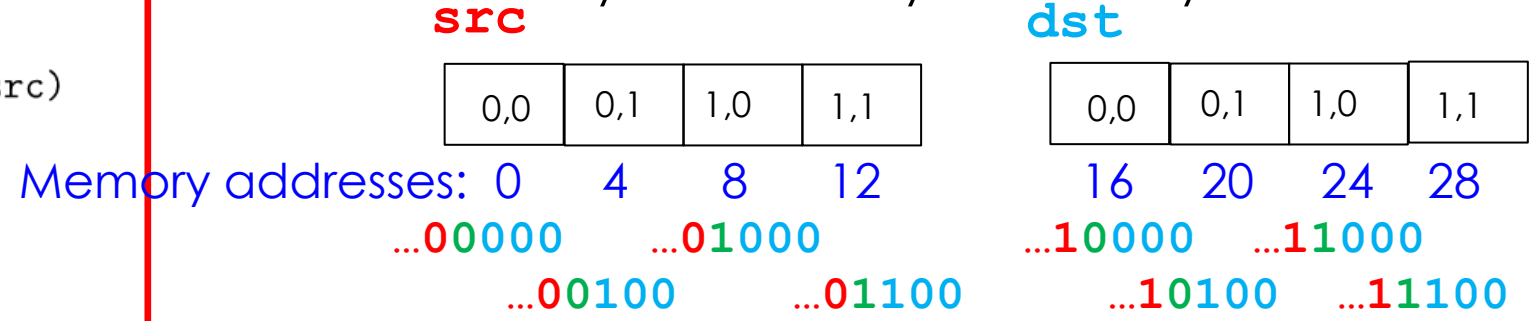
dst

	Col. 0	Col. 1
Row 0	m	
Row 1		

```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10
11
12  }
```

2nd access: 0 0	1st: 0 0
4th access: 1 0	3rd: 0 1
6th access: 0 1	5th: 1 0
8th access: 1 1	7th: 1 1
column wise	row wise

2. Draw arrays in memory + memory addresses:



3. Partition the memory address into sections:
   tag (? bits) set index (1 bit) block offset (3 bits)
4. List the pattern of indices as elements are accessed by microprocessor => identify the **stride**!
5. Draw the cache …
6. Execute the code and update the cache:

	v	Tag	Block		v	Tag	Block
Set 0	1	...0	src 0,0 src 0,1	Set 1	1	...0	src 1,0 src 1,1

The answer is:

src

	Col. 0	Col. 1
Row 0	m	
Row 1		

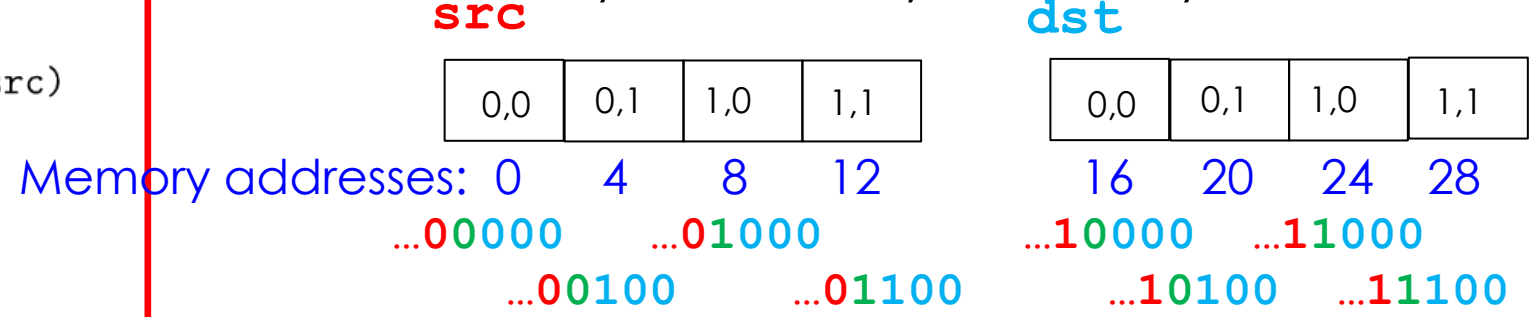
dst

	Col. 0	Col. 1
Row 0	m	
Row 1		

```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10
11
12  }
```

2nd access: 0 0	1st: 0 0
4th access: 1 0	3rd: 0 1
6th access: 0 1	5th: 1 0
8th access: 1 1	7th: 1 1
column wise	row wise

2. Draw arrays in memory + memory addresses:



3. Partition the memory address into sections:
   tag (? bits) set index (1 bit) block offset (3 bits)
4. List the pattern of indices as elements are accessed by microprocessor => identify the **stride**!
5. Draw the cache …
6. Execute the code and update the cache:

	v	Tag	Block		v	Tag	Block
Set 0	1	...1	dst 0,0 dst 0,1	Set 1	1	...0	src 1,0 src 1,1

The answer is:

src

	Col. 0	Col. 1
Row 0	m	
Row 1		

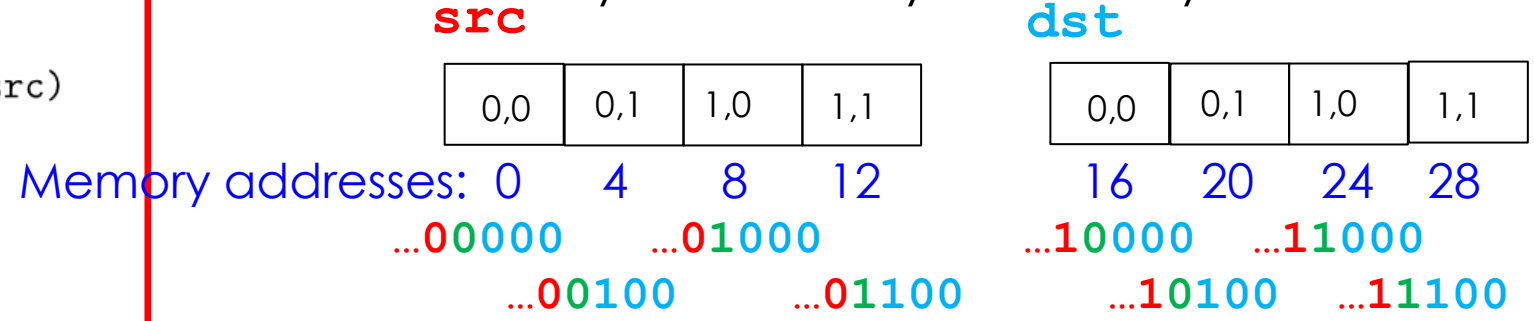
dst

	Col. 0	Col. 1
Row 0	m	
Row 1		

```
1   typedef int array[2][2];
2
3   void transpose1(array dst, array src)
4   {
5       int i, j;
6
7       for (i = 0; i < 2; i++) {
8           for (j = 0; j < 2; j++) {
9               dst[j][i] = src[i][j];
10
11
12  }
```

2nd access: 0 0	1st: 0 0
4th access: 1 0	3rd: 0 1
6th access: 0 1	5th: 1 0
8th access: 1 1	7th: 1 1
column wise	row wise

2. Draw arrays in memory + memory addresses:



3. Partition the memory address into sections:
   tag (? bits) set index (1 bit) block offset (3 bits)
4. List the pattern of indices as elements are accessed by microprocessor => identify the **stride**!
5. Draw the cache …
6. Execute the code and update the cache:

	v	Tag	Block		v	Tag	Block
Set 0	1	…1	dst 0,0 dst 0,1	Set 1	1	…0	src 1,0 src 1,1

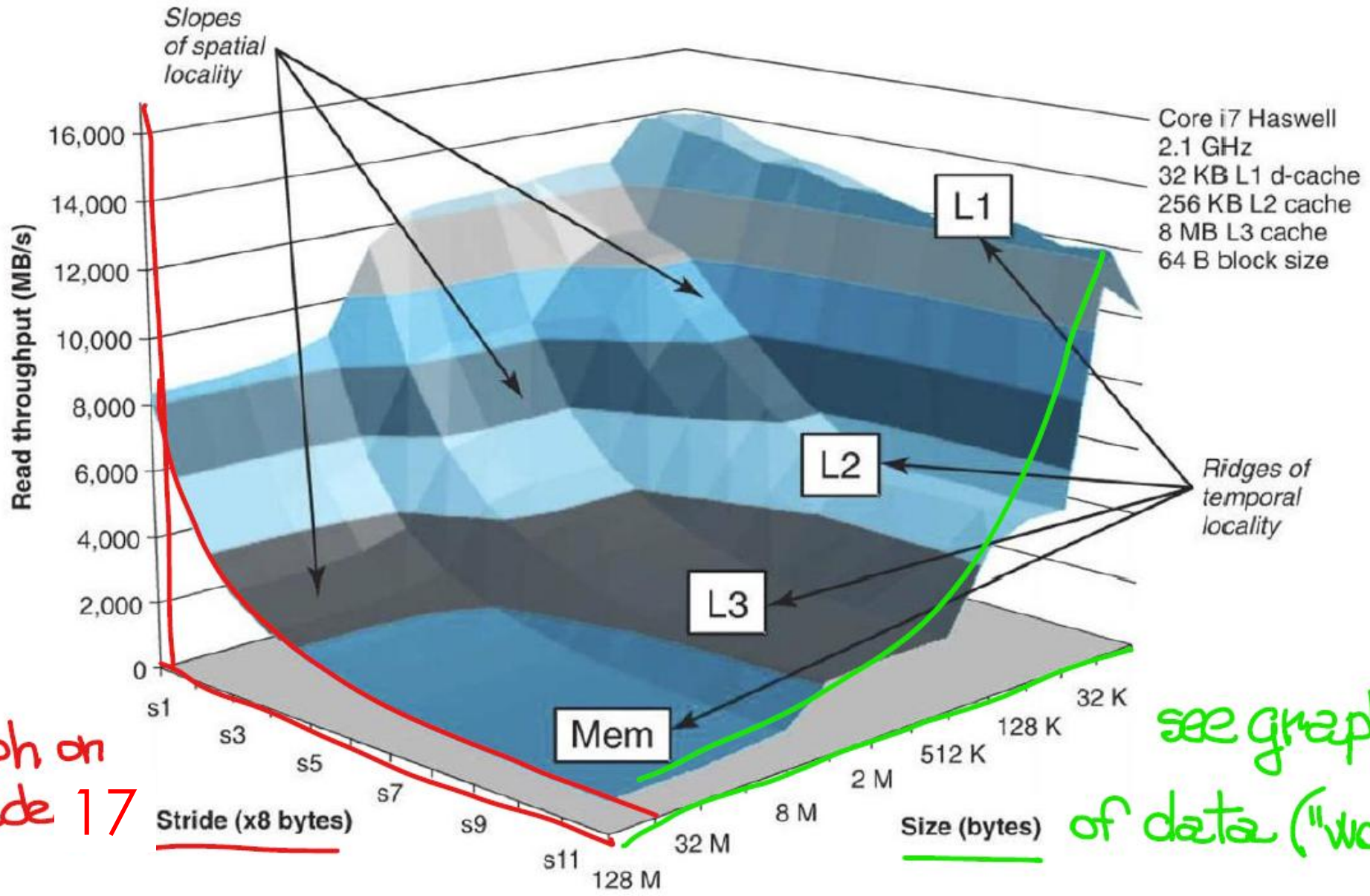
The answer is:

src

	Col. 0	Col. 1
Row 0	m	
Row 1		

dst

	Col. 0	Col. 1
Row 0	m	
Row 1		



see graph on Slide 17

see graph on Slide 18 of data ("working set")

Figure 6.41 **A memory mountain.** Shows read throughput as a function of temporal and spatial locality.

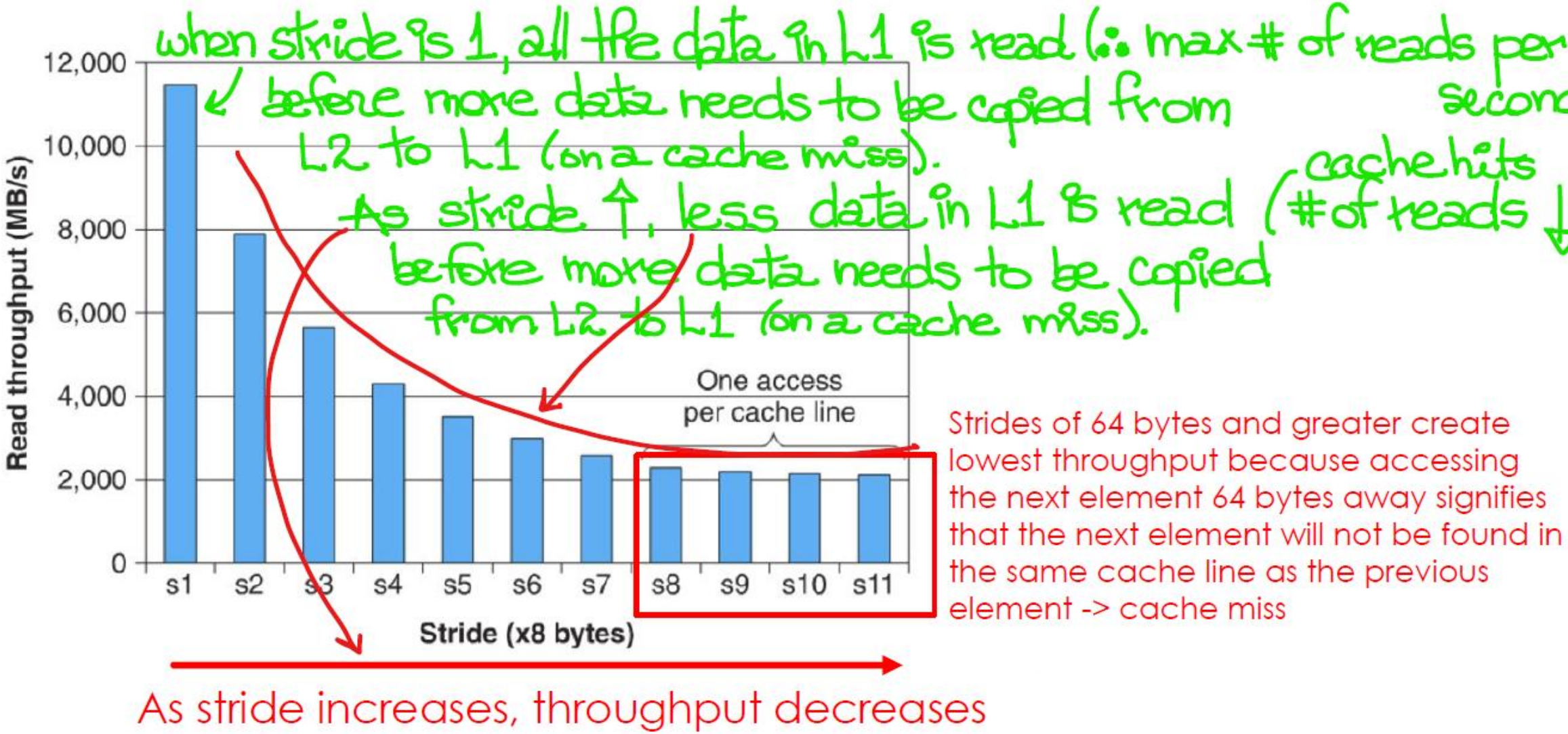


Figure 6.43 **A slope of spatial locality.** The graph shows a slice through Figure 6.41 with `size` = 4 MB.

Ridges of memory mountain follow cache limits: 32 KB, 256 KB, 8 MB

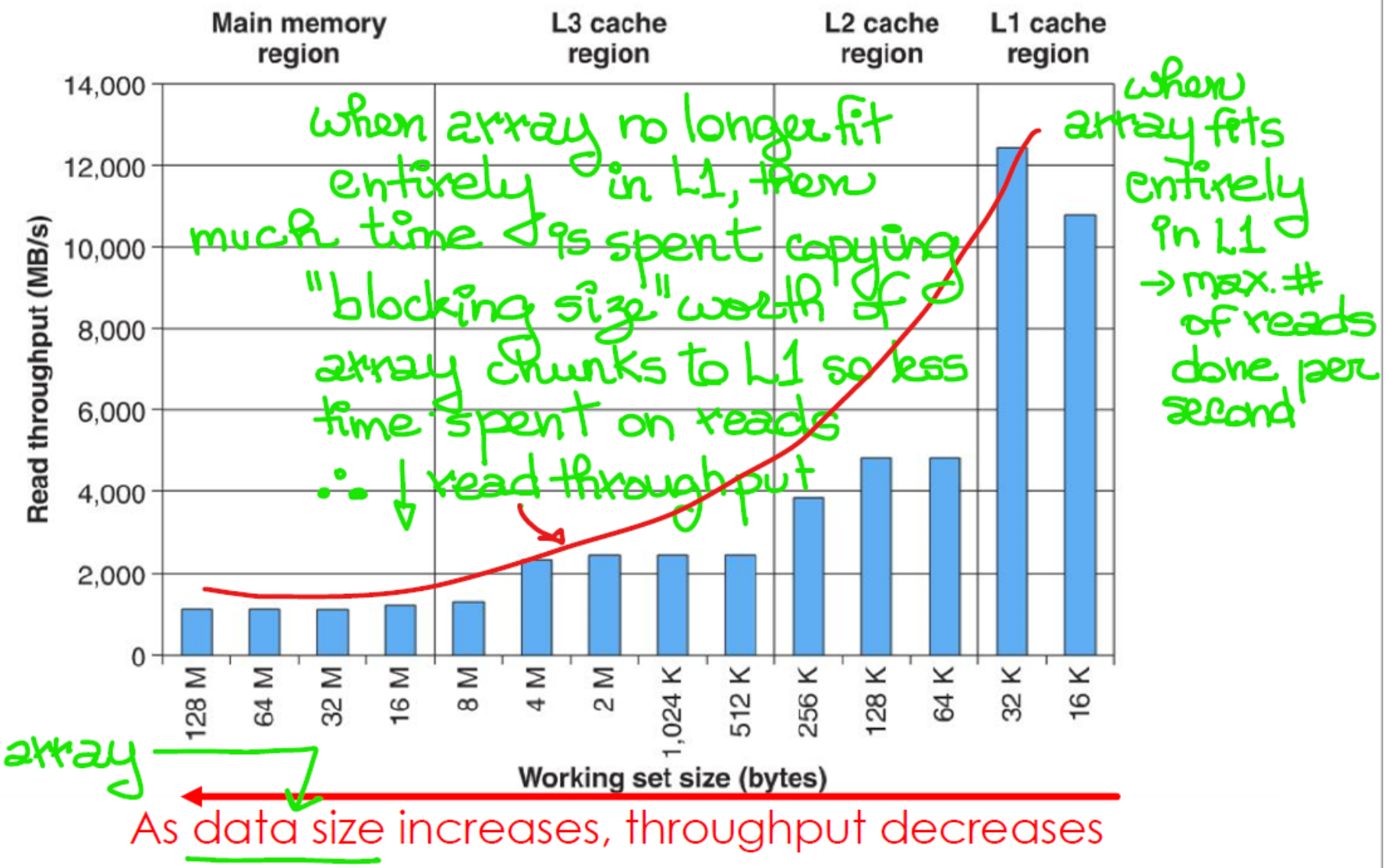


Figure 6.42 **Ridges of temporal locality in the memory mountain.** The graph shows a slice through Figure 6.41 with `stride = 8`.

Summary

- Understanding ***locality***, ***memory hierarchy*** and ***caching*** allows us to write ***cache-friendly code***
  - Programs with ***good locality*** tend to run faster because such code maximizes the use of data/instructions found in cache, i.e., such code maximizes the need to access either same memory location and/or nearby locations
- Writing ***cache-optimized code***
  - The basic idea of cache optimizations is to use all the data in a block (***spatial locality***) repeatedly (***temporal locality***) before it is replaced on a ***cache miss***
  - Reuse data already in the cache -> ***temporal locality*** hence lowering miss rates

No next lecture ☺

- Final Exam -> Friday August 15
  - See ***Information about our Final Exam*** on our course web site
- Office hours during Week 14 – Week of Monday August 11

Day	Time	Location	Person
Monday August 11	From 3pm to 4:30pm	CSIL - ASB 9838N - Large labroom in the centre of CSIL	Anne
Tuesday August 12	From 11am to 1pm	CSIL - ASB 9838N - Large labroom in the centre of CSIL	Mo
Wed. August 13	From 1pm to 2:30pm	CSIL - ASB 9838N - Large labroom in the centre of CSIL	Anne
Wed. August 13	From 4pm to 6pm	CSIL - ASB 9838N - Large labroom in the centre of CSIL	Sedi
Thursday August 14	From 2pm to 4pm	Zoom link is https://sfu.zoom.us/j/82660485101 (Meeting ID: 826 6048 5101, no passcode)	Sitong